Task 1

Task 1.1: PyTorch Tensor

```
import torch
import numpy as np
import torch.nn as nn
import torch.nn.functional as F
```

```
# Creatng Tensors
# Integer tensor
a = torch.tensor([1, 2, 3, 4])

# Float tensor
b = torch.tensor([1.5, 2.5, 3.5])

# Random tensor
c = torch.rand(3, 3)

# Zero tensor
d = torch.zeros(2, 2)

# Ones tensor
e = torch.ones(2, 3)

print("Integer Tensor:\n", a)
print("\nFloat Tensor:\n", b)
print("\nRandom Tensor:\n", c)
print("\nZero Tensor:\n", d)
print("\nOnes Tensor:\n", e)
```

```
Integer Tensor:
 tensor([1, 2, 3, 4])

Float Tensor:
 tensor([1.5000, 2.5000, 3.5000])

Random Tensor:
 tensor([[0.3472, 0.7146, 0.2333],
        [0.6035, 0.9585, 0.3183],
        [0.6037, 0.0635, 0.0607]])

Zero Tensor:
 tensor([[0., 0.],
        [0., 0.]])

Ones Tensor:
 tensor([[1., 1., 1.],
        [1., 1., 1.]])
```

```
# Tensors Shapes and data types

print("Tensor:\n", a)
print("Shape:", a.shape)
print("Dimensions:", a.ndim)
print("Data Type:", a.dtype)
print("\n")
print("Tensor:\n", e)
print("Shape:", e.shape)
print("Dimensions:", e.ndim)
print("Data Type:", e.dtype)
print("\n")
```

```
Tensor:
 tensor([1, 2, 3, 4])
Shape: torch.Size([4])
Dimensions: 1
Data Type: torch.int64


Tensor:
 tensor([[1., 1., 1.],
```

```
        [1., 1., 1.]])
Shape: torch.Size([2, 3])
Dimensions: 2
Data Type: torch.float32
```

```
# Reshaping the Tensor
reshaped = e.reshape(3,2)

print("\nReshaped:")
print(reshaped)
```

```
Reshaped:
tensor([[1., 1.],
        [1., 1.],
        [1., 1.]])
```

```
# Flatten the Tensor
flat = reshaped.view(-1)

print(flat)
```

```
tensor([1., 1., 1., 1., 1., 1.])
```

```
# Tensor Indexing

print("\nFirst Row:")
print(c[0])

print("\nSecond Column:")
print(c[:,1])

print("\nElement:")
print(c[2,2])
```

```
First Row:
tensor([0.3472, 0.7146, 0.2333])

Second Column:
tensor([0.7146, 0.9585, 0.0635])

Element:
tensor(0.0607)
```

```
# Tensor Slicing
print(c,"\n")
print(c[0:2,0:1])
```

```
tensor([[0.3472, 0.7146, 0.2333],
        [0.6035, 0.9585, 0.3183],
        [0.6037, 0.0635, 0.0607]])

tensor([[0.3472],
        [0.6035]])
```

```
# Basic Math Operations
A = torch.tensor([[1.,2.],
                  [3.,4.]])

B = torch.tensor([[5.,6.],
                  [7.,8.]])

print("Addition\n",A+B)
print("\nSubtraction\n",A-B)
print("\nMultiplication\n",A*B)
print("\nDivision\n",A/B)
```

```
Addition
 tensor([[ 6.,  8.],
        [10., 12.]])

Subtraction
 tensor([[-4., -4.],
        [-4., -4.]])
```

```
Multiplication
 tensor([[ 5., 12.],
        [21., 32.]])

Division
 tensor([[0.2000, 0.3333],
        [0.4286, 0.5000]])
```

```
# Matrics Operations

torch.matmul(A,B)
```

```
tensor([[19., 22.],
        [43., 50.]])
```

```
print("Mean:", A.mean())
print("Sum:", A.sum())
print("Maximum:", A.max())
print("Minimum:", A.min())
```

```
Mean: tensor(2.5000)
Sum: tensor(10.)
Maximum: tensor(4.)
Minimum: tensor(1.)
```

```
tensor = torch.rand(3,3)

# Define the device
device = 'cuda' if torch.cuda.is_available() else 'cpu'

tensor = tensor.to(device)

print(tensor)
print("Device:", tensor.device)
```

```
tensor([[0.2491, 0.3627, 0.1670],
        [0.6519, 0.9279, 0.4750],
        [0.9690, 0.1869, 0.2685]])
Device: cpu
```

```
# Automatic Differentiation (Autograd)
x = torch.tensor(2.0, requires_grad=True)

y = x**2 + 3*x + 1

y.backward()

print("x =", x.item())
print("y =", y.item())
print("dy/dx =", x.grad.item())
```

```
x = 2.0
y = 11.0
dy/dx = 7.0
```

```
# Multi-variable Gradient
x = torch.tensor(2.0, requires_grad=True)
y = torch.tensor(3.0, requires_grad=True)

z = x*y + x**2

z.backward()

print("dz/dx =", x.grad.item())
print("dz/dy =", y.grad.item())
```

```
dz/dx = 7.0
dz/dy = 2.0
```

```
# Disable Gradient Tracking
with torch.no_grad():
    a = torch.tensor([2.0])
    b = a*5

print(b)
```

```
tensor([10.])
```

```
# NumPy Array
np_array = np.array([[1,2],
                     [3,4]])

print(np_array)
```

```
[[1 2]
 [3 4]]
```

```
# Convert NumPy to PyTorch
torch_tensor = torch.from_numpy(np_array)

print(torch_tensor)
```

```
tensor([[1, 2],
        [3, 4]])
```

```
# Convert PyTorch to NumPy
back_numpy = torch_tensor.numpy()

print(back_numpy)
```

```
[[1 2]
 [3 4]]
```

```
# Compare NumPy and PyTorch Operations
np1 = np.array([1,2,3])
np2 = np.array([4,5,6])

torch1 = torch.tensor([1,2,3])
torch2 = torch.tensor([4,5,6])

print("NumPy Addition:")
print(np1 + np2)

print("\nPyTorch Addition:")
print(torch1 + torch2)

print("\nNumPy Multiplication:")
print(np1 * np2)

print("\nPyTorch Multiplication:")
print(torch1 * torch2)
```

```
NumPy Addition:
[5 7 9]

PyTorch Addition:
tensor([5, 7, 9])

NumPy Multiplication:
[ 4 10 18]

PyTorch Multiplication:
tensor([ 4, 10, 18])
```

Task 1.2: Build Your First CNN

```
# CNN Architecture

class SimpleCNN(nn.Module):
    def __init__(self, num_classes=10):
        super(SimpleCNN, self).__init__()

        # Convolutional Layer 1
        # Input: 3 channels (RGB image)
        # Output: 16 feature maps
        self.conv1 = nn.Conv2d(
            in_channels=3,
            out_channels=16,
            kernel_size=3,
            padding=1
```

```
        )

        # Convolutional Layer 2
        # Input: 16 channels
        # Output: 32 feature maps
        self.conv2 = nn.Conv2d(
            in_channels=16,
            out_channels=32,
            kernel_size=3,
            padding=1
        )

        # Max Pooling Layer (2x2)
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # Fully Connected Layer 1
        # After two pooling layers, CIFAR-10 image size becomes:
        # 32x32 → 16x16 → 8x8
        # Feature maps = 32
        self.fc1 = nn.Linear(32 * 8 * 8, 128)

        # Fully Connected Layer 2 (Output layer)
        self.fc2 = nn.Linear(128, num_classes)

        # Dropout layer for regularization
        self.dropout = nn.Dropout(0.5)

    def forward(self, x):
        # ----- First Convolution Block -----
        # Conv → ReLU → Pool
        x = self.pool(F.relu(self.conv1(x)))

        # ----- Second Convolution Block -----
        # Conv → ReLU → Pool
        x = self.pool(F.relu(self.conv2(x)))

        # ----- Flatten -----
        x = x.view(-1, 32 * 8 * 8)

        # ----- Fully Connected Layers -----
        x = F.relu(self.fc1(x))
        x = self.dropout(x)
        x = self.fc2(x)

        return x
```

```
# Output Dimensions
print("CNN Output Dimension Calculation")
print("-" * 40)

print("Input image: 3 x 32 x 32")
print("After conv1: 16 x 32 x 32")
print("After pool1: 16 x 16 x 16")
print("After conv2: 32 x 16 x 16")
print("After pool2: 32 x 8 x 8")
print("Flatten: 32 * 8 * 8 = 2048")
print("FC1: 2048 → 128")
print("FC2: 128 → 10")
```

```
CNN Output Dimension Calculation
----------------------------------------
Input image: 3 x 32 x 32
After conv1: 16 x 32 x 32
After pool1: 16 x 16 x 16
After conv2: 32 x 16 x 16
After pool2: 32 x 8 x 8
Flatten: 32 * 8 * 8 = 2048
FC1: 2048 → 128
FC2: 128 → 10
```

```
# Creating the Model
model = SimpleCNN(num_classes=10).to(device)

print("Model created successfully!")
```

```
Model created successfully!
```

```
# Model Architecture
print(model)
```

```
SimpleCNN(
  (conv1): Conv2d(3, 16, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(16, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (fc1): Linear(in_features=2048, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
  (dropout): Dropout(p=0.5, inplace=False)
)
```

```
# Total Parameters
total_params = sum(p.numel() for p in model.parameters())
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)

print("Total Parameters:", total_params)
print("Trainable Parameters:", trainable_params)
```

```
Total Parameters: 268650
Trainable Parameters: 268650
```

```
# Parameter Shapes
print("Model Parameters and Shapes")
print("-" * 50)

for name, param in model.named_parameters():
    print(f"{name:20} {str(param.shape):20} {param.numel()} params")
```

```
Model Parameters and Shapes
--------------------------------------------------
conv1.weight         torch.Size([16, 3, 3, 3]) 432 params
conv1.bias           torch.Size([16])     16 params
conv2.weight         torch.Size([32, 16, 3, 3]) 4608 params
conv2.bias           torch.Size([32])     32 params
fc1.weight           torch.Size([128, 2048]) 262144 params
fc1.bias             torch.Size([128])    128 params
fc2.weight           torch.Size([10, 128]) 1280 params
fc2.bias             torch.Size([10])     10 params
```

```
# Test with Random Input
# Batch size = 4
# Channels = 3 (RGB)
# Height = 32
# Width = 32

random_input = torch.randn(4, 3, 32, 32).to(device)

print("Random input shape:", random_input.shape)
```

```
Random input shape: torch.Size([4, 3, 32, 32])
```

```
# Forward Pass
output = model(random_input)

print("Output shape:", output.shape)
print("Output tensor:")
print(output)
```

```
Output shape: torch.Size([4, 10])
Output tensor:
tensor([[ 0.0854, -0.0847,  0.0889,  0.0024,  0.1147,  0.0048, -0.0832,  0.1478,
         -0.0175,  0.1232],
        [-0.0101,  0.1114,  0.0611,  0.0756,  0.0590, -0.0074,  0.2770,  0.1610,
          -0.0802, -0.0087],
        [ 0.1579,  0.1195,  0.2806, -0.1903, -0.0112,  0.1456, -0.1003, -0.0046,
          -0.1590,  0.0563],
        [ 0.1331,  0.0801,  0.1176,  0.1262, -0.0523, -0.0040,  0.0462,  0.1870,
          -0.0505, -0.0152]], grad_fn=<AddmmBackward0>)
```

```
# Output Shape
print("Expected output shape: (4, 10)")
print("Actual output shape:", output.shape)

if output.shape == (4, 10):
    print("Output shape is correct!")
```

```
else:
    print("Output shape is incorrect")
```

```
Expected output shape: (4, 10)
Actual output shape: torch.Size([4, 10])
Output shape is correct!
```

```
# Test with Different Batch Sizes
batch_sizes = [1, 2, 8]

for batch_size in batch_sizes:
    test_input = torch.randn(batch_size, 3, 32, 32).to(device)
    test_output = model(test_input)

    print(f"Input batch size: {batch_size}")
    print(f"Output shape: {test_output.shape}")
    print("-" * 30)
```

```
Input batch size: 1
Output shape: torch.Size([1, 10])
------------------------------
Input batch size: 2
Output shape: torch.Size([2, 10])
------------------------------
Input batch size: 8
Output shape: torch.Size([8, 10])
------------------------------
```

```
# Analyze Layer Outputs
x = torch.randn(1, 3, 32, 32).to(device)

print("Input:", x.shape)

# Conv1
x = model.conv1(x)
print("After conv1:", x.shape)

# ReLU + Pool1
x = model.pool(F.relu(x))
print("After pool1:", x.shape)

# Conv2
x = model.conv2(x)
print("After conv2:", x.shape)

# ReLU + Pool2
x = model.pool(F.relu(x))
print("After pool2:", x.shape)

# Flatten
x = x.view(-1, 32 * 8 * 8)
print("After flatten:", x.shape)

# FC1
x = model.fc1(x)
print("After fc1:", x.shape)

# FC2
x = model.fc2(F.relu(x))
print("After fc2:", x.shape)
```

```
Input: torch.Size([1, 3, 32, 32])
After conv1: torch.Size([1, 16, 32, 32])
After pool1: torch.Size([1, 16, 16, 16])
After conv2: torch.Size([1, 32, 16, 16])
After pool2: torch.Size([1, 32, 8, 8])
After flatten: torch.Size([1, 2048])
After fc1: torch.Size([1, 128])
After fc2: torch.Size([1, 10])
```